

One Agent → Lightweight → Workload & Mode

CPU usage:- 1-10%

Low traffic - 1-2%

Memory Usage:- 200MB - 500MB

Deep Monitoring - 500MB - 1GB

Disk - 100MB - 500MB

Fullstack - Highest

Infra. - Medium

Discovery - Lowest

Java/.Net (Deep tracing) → Higher usage

Single process - Low usage

Traffic Volume - More req. ↑ more traces ↑ more CPU/memory

Feature enabled

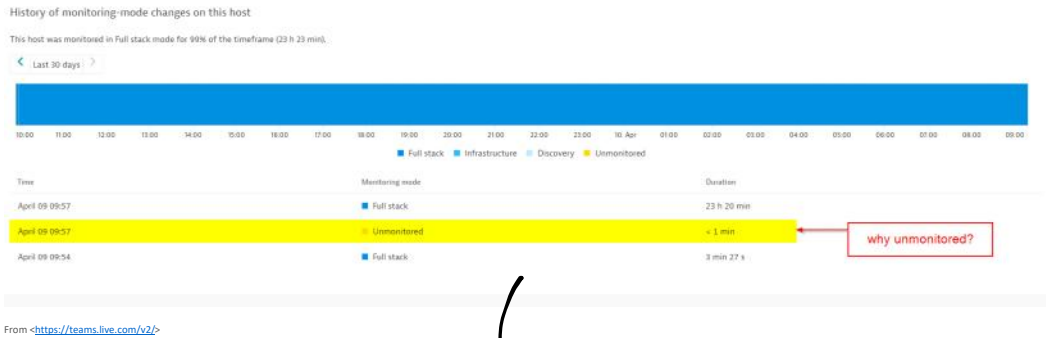
① Path

② Log

③ KES

④ Params (API) Correlation

- ① Java/.Net < 5% CPU
- ② Memory - 200MB - 1GB
- ③ Minimal N/w impact.



One Agent temporarily lost monitoring control

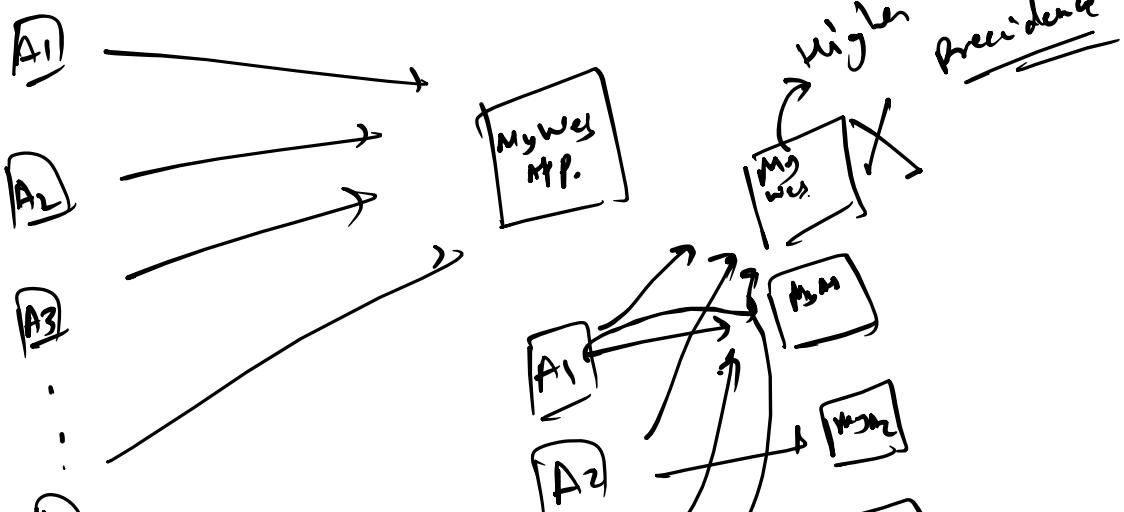
- ① Restart / update
- ② Host / VM level event → OS reboot
Network glitch
CPU spike.

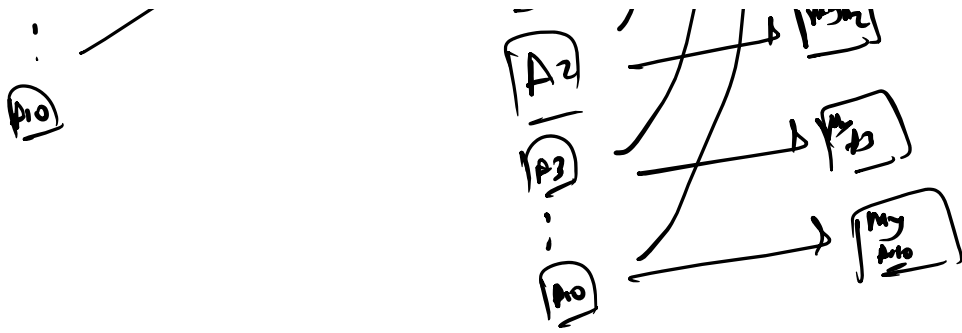
☐ CodeModule injection

Inject CodeModules in Discovery mode.

From <<https://teams.live.com/v2/>>

Host Monitoring → APM





XHR Action:-

→ (XML HTTP Request)

→ Background API calls made by the Browser
(ASAX / Fetch Req.)

Call Backend APIs in the Background

Background Calls = XHR Action

Action:-

- ① Page Load → Full page refresh
- ② XHR Action → Background API call
- ③ Custom Action → User-Defined.

IHP (Interaction to Next point):-

click } How quickly a webpage responds to
Tap } user interaction!
Key Press }

User Click → Processing → UI update

IHP

Use Click

JMP

Need Input
Poor

Good $\rightarrow < 200ms$
 $\rightarrow 200 - 500ms$
 $\rightarrow > 500ms$

why?

- ① Slow Button response
- ② Laggy UI
- ③ Delayed typing.

What?

- ① Heavy JS execution
- ② main thread blockage
- ③ Large DOM updates

- ④ Slow API response.
- ⑤ 3rd party scripts

User click $\rightarrow$ JS $\rightarrow$ XHR/AJAX $\rightarrow$ Response $\rightarrow$ UI update.

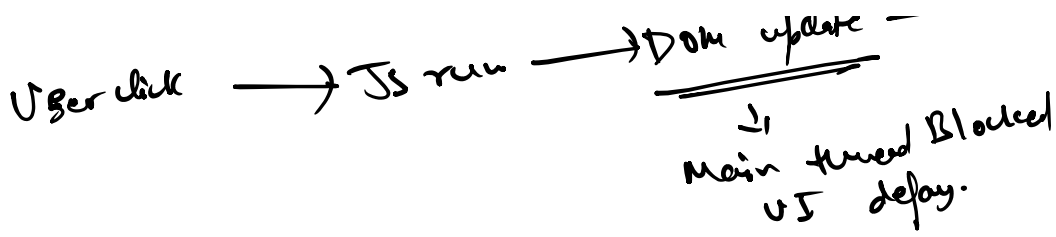
Large DOM update:-

DOM - Document object Model.

$\rightarrow$ structure of your webpage (HTML elements)

$\rightarrow$ Browser is forced to update/re-render a huge no. of elements at once.

User click $\rightarrow$ JS run $\rightarrow$ DOM update $\rightarrow$ Paint.



Large DOM:-

- ① Laggy UI
- ② Scroll stutter
- ③ Delay in Button
- ④ High JMP / Loop task in browser

Technical Cause:-

- ① Too many element.
- ② Full renders (SPA issue)
- ③ Layout Threshing.
- ④ Heavy JS framework.

Composite metrics across response times

Average	metrics of	Load actions		
22.16s	17.78s	10.69s	9.95s	17.77s
User action duration	Visually complete	Speed index	DOM interactive	Load event end
17.73s	5.56s	5.36s	0.00s	3.13s
Load event start	HTML downloaded	Time to first byte	First input delay	Largest contentful paint
0.004	0.65s	4.93s		
Cumulative layout shift	Server contribution	Network contribution		

Good
 < 0.1s
 0.1 - 0.2s
 0.2 - 0.5s
 > 0.5s

→ Visual stability

User Action → user click → page fully ready

Visually Complete → when the page looks fully loaded to user.

Speed Index → Lower = better UX.

... loader timeline.

→ Delay in Interact. → Browser response
 < 2.5s = good
 3-13s need improvement

Speed Index $\rightarrow$ Lower = better UX.
Visual loading timeline.

DOM Interactive - Interactive - (JS ready.)

load event Metrics $\rightarrow$ Load event start $\rightarrow$ start loading
 $\rightarrow$ Load event end $\rightarrow$ completion

Net & Backend Metrics
Time to ^{first} Byte - (TTFB) $\rightarrow$ 5.38s
 $\rightarrow$ HTML Backend $\rightarrow$ Time to download main HTML.

Server Contribution - Inside Backend process.

Net Contribution $\rightarrow$ Net Latency + transfer.

$$\text{View per minute} = \left(\frac{\text{Total views}}{\text{Total time (in min)}} \right)$$

$$= \left(\frac{35}{60} \right) = 0.58 \text{ view/min.}$$

... error

Error:- ① HTTP error.

② JS error

③ Custom error

① Req. error:- N/w or API or resource calls

HTTP 404 →
/maps → 400 - 500 → HTTP error

② JS error:- Browser execution.

Type Error
Ref error.

1.74/min.

③ Custom error Set By Developer
throw new Error("payment failed");

One Agent

Metric → ~ 10 sec.
Trace → Near real time
Log → 5s

RUM Beacon → On user Action.
Events / broken → Immediate.

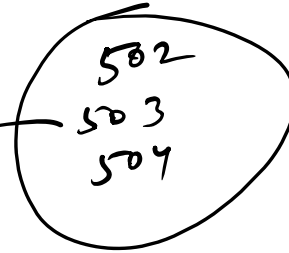
Error → 1st Party → Error By developer
→ 2nd Party → Error Done By external Vendor
→ CDN → Content Delivery Network.



User $\rightarrow$ CDN $\rightarrow$ origin Servers



CDN Not able to reach to server.



Apdex Rating:-

Score

1.0

0.85 - 0.99

0.7 - 0.84

< 0.7

Meaning

Excellent

Good

Fair

Poor

$T = 2\text{sec.}$

$RT \leq 2s$

2s - 8s

> 8s or Error

How!

① Satisfied $\rightarrow RT \leq T$

② Tolerating $\rightarrow B/w T \text{ \& } 4T$

③ Frustrated $\rightarrow > 4T \text{ or Error}$

Apdex = $(\text{Satisfied} + 0.5 \times \text{Tolerating}) \div \text{Total req.}$

100 req. $\rightarrow$ $\frac{70 \text{ Satisfied}}{20 \text{ Tolerating}}$
10 Frustrated.

$$\left(\frac{70 + 0.5 \times 20}{100} \right) = \frac{80}{100} = 0.8$$

Apdex

Why!

$\therefore 3\text{sec.}$

How?

① Reduce RT

② Reduce Error
move forward.

Why?
RT is 3sec.
Only 60% is happy

- ② Reduce
- ③ Improve frontend.

Apdex - How happy your users are with
your app performance!